

 Member-only story

# I Used Claude Code to Build a Personal Knowledge Base — Inspired by Karpathy's LLM Wiki Pattern

How a three-layer architecture turns your LLM into a librarian that actually keeps the shelves organized



Mark Chen

Follow

6 min read · Apr 6, 2026



Last weekend I stumbled on a gist from Andrej Karpathy describing something he called the “LLM Wiki” — a pattern where language models don’t just answer your questions, they maintain a living knowledge base on your behalf. I read it, felt the quiet click of recognition, and thought: I want this for the two domains I work in every week.

An hour later, I had it. Two structured wikis — one for writing on Medium, another for BI reporting with SQL, Python, and Power BI — built entirely through a conversation with Claude Code.

Here’s what the pattern is, why it matters, and how you can build one yourself.

## What Karpathy Proposed

The LLM Wiki pattern rests on a simple architectural insight. Most of us use AI the same way: ask a question, get an answer, close the tab. Tomorrow, we ask a slightly different version of the same question. The knowledge never compounds.

Karpathy's alternative has three layers:

**Raw sources** — immutable documents you curate. Articles you've read, code snippets that worked, analytics exports, conference notes. These go in and never change.

**The wiki** — markdown pages the LLM generates and maintains. Summaries, patterns, cross-references, working examples. This is the layer that grows.

**The schema** — a configuration document that tells the LLM how to behave: naming conventions, page structure, what operations to perform, who's responsible for what.

Two supporting files tie everything together: an `index.md` for categorical discovery and a `log.md` that tracks every change chronologically.

The key shift is philosophical. Instead of retrieving from raw documents each time — the standard RAG approach — the LLM incrementally builds structured knowledge. Each new source gets woven into existing pages, cross-references get updated, contradictions get flagged. The wiki remembers so you don't have to.

## Why This Resonated With Me

I write about two things: the craft of blogging (particularly on Medium) and the craft of building BI reports. Both domains share a frustrating pattern — I keep rediscovering things I already learned.

How does Medium's algorithm weight read ratio versus claps? I've looked this up four times. What's the cleanest DAX pattern for year-over-year growth?

I've written it, forgotten the SAMEPERIODLASTYEAR syntax, and rewritten it from scratch more times than I'd like to admit.

Traditional note-taking doesn't solve this. My notes get long, disorganized, and eventually abandoned. The maintenance burden grows faster than the value — exactly the failure mode Karpathy identifies.

The LLM Wiki flips the maintenance equation. The tedious work of summarizing, cross-referencing, and keeping things consistent is exactly what language models are good at. The human stays responsible for the interesting parts: choosing what to learn, asking the right questions, and making creative decisions.

## **What I Built in One Session**

I opened Claude Code and described what I wanted. Within a single conversation, it produced the full structure for both domains.

### **Blog Writing Wiki (6 pages)**

The schema defines the wiki's scope: content strategy, writing craft, Medium-specific mechanics, editorial workflow, and analytics. Each page follows a consistent format with YAML front-matter, wiki-style cross-links, and source citations.

The initial pages cover:

- **Medium Platform Guide** — how distribution, the Partner Program, and ranking signals actually work
- **Writing Structure and Hooks** — opening patterns, body structures (listicle, narrative, problem-solution), pacing rules
- **Medium SEO Tactics** — why Medium's domain authority matters, on-page optimization, content strategy for search

- **Content Ideation** — where ideas come from, a validation checklist, the pillar-and-satellite model
- **Editorial Workflow** — a five-stage pipeline from ideation to post-publish analytics
- **Voice and Tone** — the dinner party test, voice attributes, tone adjustments by content type

## **BI Reporting Wiki (7 pages)**

This wiki covers the full stack I use daily: SQL for querying, Python for wrangling and automation, Power BI for visualization and delivery.

The initial pages include:

- **SQL Query Patterns** — CTEs, window functions, conditional aggregation, date spines, and anti-patterns to avoid
- **SQL Performance Optimization** — EXPLAIN plans, indexing strategy, sargable predicates, materialized views
- **Data Modeling: Star Schema** — facts, dimensions, date dimension design, slowly changing dimensions
- **Python for BI** — where Python fits alongside SQL and Power BI, pandas essentials, automation patterns
- **Python-SQL Integration** — SQLAlchemy connections, parameterized queries, Jinja templating, data quality tests
- **DAX Essentials** — measures versus calculated columns, CALCULATE, time intelligence, iterator functions
- **Power BI Data Modeling** — Vertipaq optimization, Power Query folding, row-level security, deployment pipelines

Every page includes working code examples. The SQL pages have real queries. The DAX pages have real measures. The Python pages have real

pandas transformations. This isn't abstract documentation — it's a reference I can copy from.

## How the Wiki Grows

The real value isn't in the initial pages. It's in what happens next.

### Ingest

When I read something useful — an article about a new Medium algorithm change, a clever DAX pattern from a colleague, a performance tip from a conference talk — I drop it into the `raw/` folder and ask Claude Code to ingest it. The LLM reads the source, extracts takeaways, and updates the relevant wiki pages. New concepts get new pages. Existing pages get richer.

### Query

When I need to remember something, I query the wiki instead of searching the internet. The LLM synthesizes an answer from existing pages, with citations back to the sources. If the answer is substantial enough to be reusable, it becomes a new wiki page automatically.

### Lint

Periodically, I ask the LLM to health-check the wiki. It looks for contradictions between pages, outdated advice (Medium changes features constantly), orphaned pages with no inbound links, and gaps where a topic is referenced but no page exists yet.

## The Division of Labor

What makes this sustainable is a clear division of responsibility:

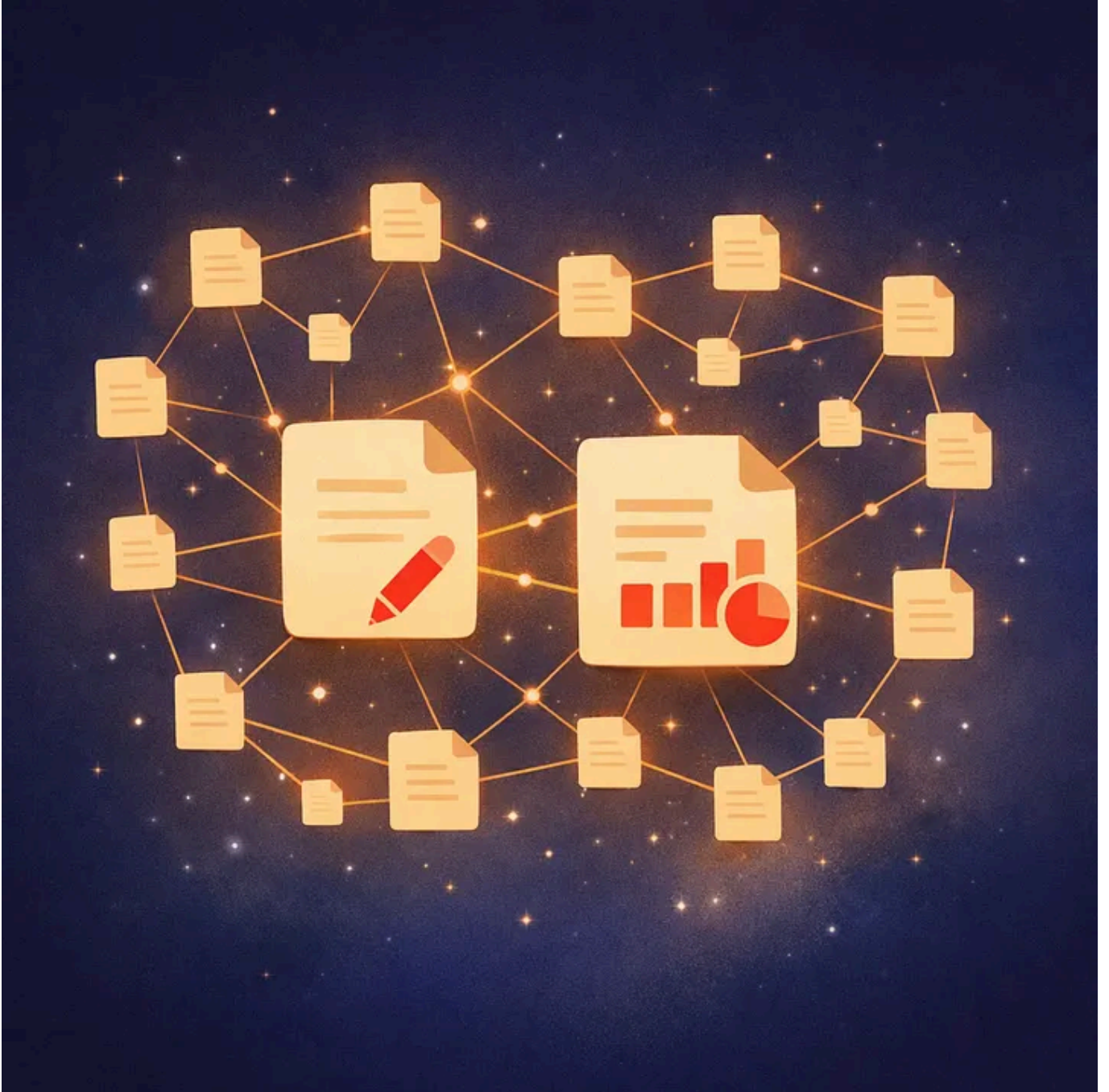
|     |         |                          |                              |                        |
|-----|---------|--------------------------|------------------------------|------------------------|
| You | The LLM | Choose what to learn     | Summarize and structure      | Set creative direction |
|     |         | Cross-reference and link | Make judgment calls          | Maintain consistency   |
|     |         | Decide what to publish   | Flag gaps and contradictions |                        |

You stay in the driver's seat. The LLM handles the bookkeeping you'd never do yourself.

## Getting Started

If you want to try this yourself, here's the minimum viable structure:

```
my-wiki/
├── schema.md    ← rules and conventions
├── index.md     ← page catalog
├── log.md       ← change history
├── raw/        ← your source material
└── wiki/       ← LLM-maintained pages
```



Write a schema that defines your domain scope, file naming conventions, and the three operations (ingest, query, lint). Then have a conversation with your preferred coding assistant and ask it to create the initial wiki pages based on what you already know.

The first session gives you a foundation. The second session, when you ingest your first external source and watch the wiki update itself, is when the pattern clicks.

## **What I'm Learning**

Three things stand out after living with this for a few days.

First, **the wiki changes how I read**. When I know there's a place for new knowledge to land, I read more attentively. I'm already thinking about which page this updates, what it contradicts, what gap it fills.

Second, **the schema is the most important file**. It's not the wiki pages — those can be regenerated. It's the schema that encodes your judgment about what matters, how things should be organized, and what the LLM should care about.

Third, **this is personal infrastructure**. It's not a product or a framework. It's a quiet practice of building up knowledge that compounds, one source at a time.

Karpathy described the LLM Wiki as a modern version of Vannevar Bush's Memex — a device imagined in 1945 for storing and retrieving everything a person has ever read. We've been waiting 80 years for the technology to catch up with the idea.

I think it finally has.

LLM

Knowledge Management

Claude Code

Personal Knowledge System

Data Analytics



**Written by Mark Chen**

293 followers · 106 following

Follow

I build things, break things, and write about the messy middle where humans and AI learn to get along. Part tech hobbyist, part philosopher, without degree.



# No responses yet



Christian Broberg

What are your thoughts?

---

